

AI Mock Interviewer for Data Analytics/Science

Charishma Tamarana, Kanchan Ashok Naik, Pranav Reveendran, Priya Govindarajulu, Soumya Challuru

Sreenivas, Vaishnavi Nanduri

Department of Applied Data Science, San Jose State University

Data 298B: MSADI Project II

Dr. Simon Shim

Sep 29, 2025

4. Model Development

4.1 Model Proposals

This AI mock interview system aims to provide a realistic and adaptive interview experience that surpasses traditional, static interview practices and question banks. The system integrates multiple modules to provide a realistic interview experience to the users, which includes building a front end where the user can interact with the interviewer using Voice/Text answers inputs, to the backend, where fine-tuning the models to behave like an interview coach to receive user answers, review, and then provide feedback. The data obtained from users, such as metadata and interview answers, is sent through fine-tuned large language models (LLMs).

The advanced machine learning models, especially the large language models such as Llama-3-70B-Instruct, Mistral-7B-Instruct-v0.2, Gemma-7B-IT, and Qwen2.5-7B-Instruct, are chosen for these tasks based on their unique strengths. These models perform key tasks, such as generating interview questions and providing feedback on the user's answers, along with the suggested answers generated by the models. All four models are fine-tuned on the same dataset to compare their tradeoffs in terms of reasoning power and efficiency, and to select the one that is best suited for this AI mock interview system through systematic performance metrics and alignment with this system's goals.

4.1.1 Llama-3-70B-Instruct

The Large Language Model Meta AI (LLaMA) is one of the open and efficient large language models that represents Meta's contribution to the field. This LLaMA model is a transformer-based language model to predict the next token in a sequence with improved efficiency, scalability, and openness. The LLaMA-1 is the first version introduced in 2023, and this model was based on the original transformer architecture by Vaswani et al. (2017), but included the new modifications to improve the efficiency and stability of the base transformer model. Figure 25 illustrates how the architecture of

LLaMA-1 was enhanced from the original transformer decoder block, specifically how it transitioned from post-normalization to pre-normalization to strengthen and stabilize model training. In the original transformer architecture, post-normalization is applied to each layer, meaning that residual connections are added first, followed by layer normalization. This method works well for small architectures and shows training instabilities for the deeper models with higher parameters. In LLaMA-1, the architecture is updated to pre-normalization, where each input sublayer is normalized before the feed-forward step. The SwiGlu activation is used in the feed-forward network, which captures more dependencies than the original model's ReLU activations. This LLaMA-1 architecture provides a foundation for scaling up to large models, such as LLaMA-3.

Figure 25

Llama1 model architecture(right) evolution from the original transformer architecture(left). Adapted from Roque (2025).

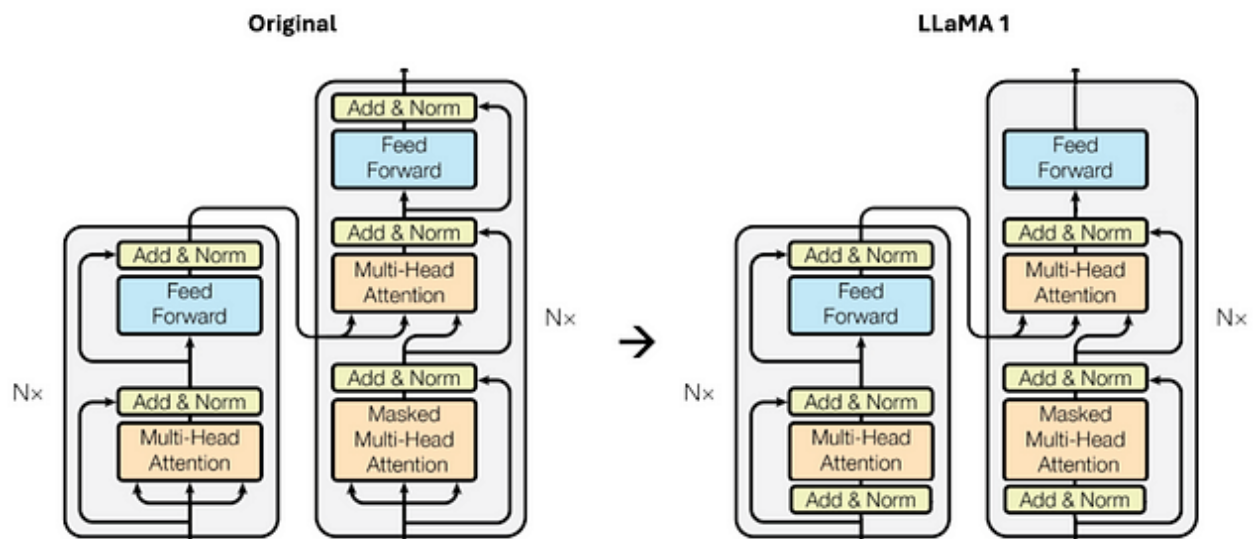
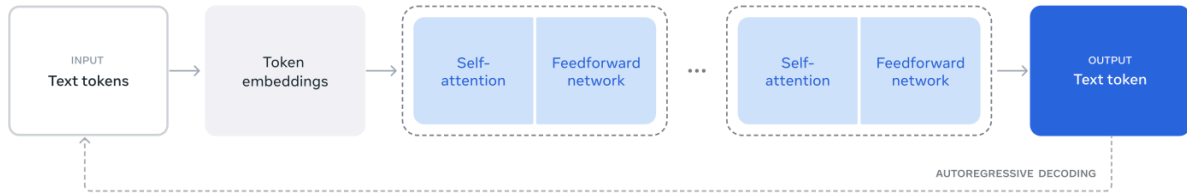


Figure 26 illustrates the overall architecture of LLaMA 3, which is based on LLaMA-1 as its foundation, achieved by scaling up the training parameters, utilizing a long context, and incorporating alignment techniques. The LLaMA 3 training increased to 15-16.5T tokens and utilizes the SwiGLU activation, optimized for long context stability. The end-to-end process of LLaMA 3 development is explained in Figure 26, which illustrates how LLaMA 3 surpasses regular pre-training by incorporating additional steps, such as long-context adaptation and model optimization for specific tasks, including instruction tuning. The LLaMA-3-70B-Instruct is an example of an instruction-tuning model that can generate interview questions and provide accurate, continuous feedback.

Figure 26

Overall architecture of Llama 3. Adapted from Grattafiori et al. (2024, p. 3).



The LLaMA 3 team correlates the amount of compute available with the dataset size required for training, and this relationship is a scaling law. This scaling law is represented as:

$$N^*(C) = AC^\alpha \quad (1)$$

Where C represents the compute budget and $N^*(C)$ represents the ideal number of training tokens. The parameters A and α are estimated empirically as $A = 0.29$ and $\alpha = 0.53$. The authors use this formula to determine that the 405B parameter model requires 16.5 trillion tokens for training, ensuring the model is computationally efficient (Grattafiori et al., 2024).

This scaling law explains that model quality depends on model size, compute budget, and data size, and confirms that the model can scale up to 128k context length without reducing the model quality. For this AI mock interview system, which combines multiple Q&A examples with feedback, the LLaMA 3 architecture is beneficial, especially for instruction-tuned alignment, stable training, and long-context capability. By combining these features with scaling and using the LoRA adapter, which requires less compute time, the LLaMA-3 provides greater support for fine-tuning and building an effective AI interview system.

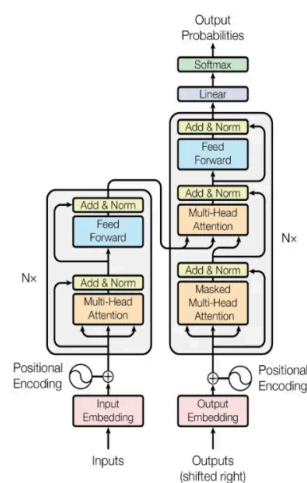
4.1.2 Mistral-7B-Instruct-v0.2

The Mistral AI introduced the Mistral-7B open source foundation model, and the work was published by Jiang et al. (2023). The goal of building this model is to create a computationally lightweight model that provides high-quality results. Mistral 7B is a decoder-only and transformer-based language model built with 7 billion parameters. It is designed to achieve better performance than models that are three or four times its size, with added new features for memory management and attention mechanisms. Like the LLaMA model, this Mistral model uses the same pre-normalization technique and SiLU activation function. Along with these features, the model introduced the Sliding Window Function and Rolling Key-Value Cache to provide long context with less memory.

Figure 27

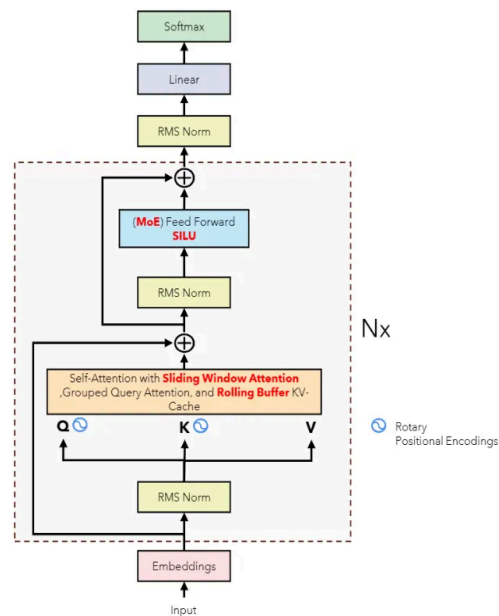
Comparison of original transformer architecture vs Mistral 7b. Adapted from Vaswani et al. (2017) and Chamabouabd (2023).

Transformer vs Mistral



Transformer

("Attention is all you need")



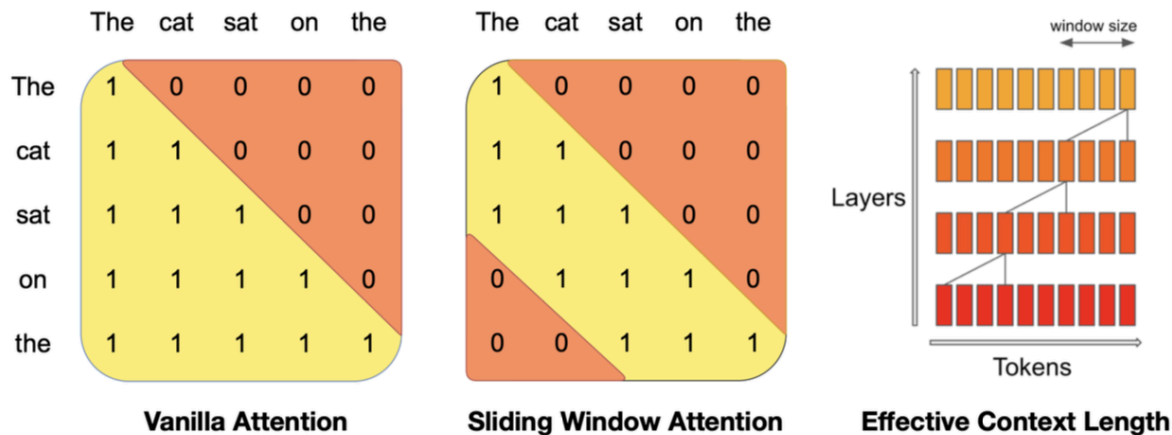
Mistral

Figure 27 explains the overall architecture of Mistral 7B (right) by comparing it to the original transformer model (left) by Vaswani et al. (2017). The transformer model utilizes full multi-head attention and post-normalization, which are prone to the high cost and instability associated with large language models. Mistral modifies this architecture by introducing several enhancements, including Grouped Query Attention, Sliding Window Attention, and Rolling KV Cache. The Rolling KV cache helps maintain a constant memory footprint by utilizing a circular buffer. The grouped query attention helps to reduce the memory by sharing key/value projections among the query heads, and the sliding window attention minimizes the number of past tokens used for each new token.

The Mistral's Sliding Window Attention feature is shown in Figure 28. The figure illustrates the difference between this sliding window attention and standard vanilla attention. The vanilla attention incurs computational and memory costs because it requires each token to attend to all previous tokens. However, Mistral addresses this issue by defining the window size, allowing each token to refer to only a limited set of past tokens. This method enables the Mistral to perform long-term context reasoning with reduced memory usage.

Figure 28

Mistral's Sliding Window Attention feature explained. Adapted from Jiang et al. (2023, p. 3)



The Mistral-7B model is suitable for this AI mock interview system, providing strengths such as instruction-following capability, a balanced model size, and the ability to handle long contexts. The model size is balanced at 7B parameters, small enough to run on a modest GPU, yet it provides high-quality responses. Using the LoRA adapters, the instruction-based Mistral model can be tuned easily to adapt to conversational prompts and directions. These strengths make the Mistral-7 7B model more suitable for this AI interview system.

4.1.3 Gemma-7B-IT

In 2024, DeepMind released Gemma family models as open-source, a lighter alternative to the closed-source mini models. The Gemma model is a decoder-based transformer and is built only as a text-only model. The Gemma 7B instruction-tuned variant is available specifically for conversational and interactive tasks. The Gemma 7B model follows the transformer architecture proposed by Vaswani et al. (2017). Additionally, it introduced several improvements, including the use of RMSNorm for each input sublayer, the replacement of regular embeddings with Rotatory Positional Embeddings (RoPE), and the adoption of GeGLU activations over ReLU.

Figure 29 shows the model key parameters updated for the Gemma model, derived from the base transformer architecture adapted from Gemma Team (2024, p. 2). The number of transformer layers

increased to 28 to provide a deeper context and reasoning. The feed-forward network units also increased to 49152 hidden units for information transformation after each step. The model added 16 attention heads for caching. These design choices make Gemma 7B more powerful for dialogue and reasoning tasks, thanks to its optimized architecture.

Figure 29

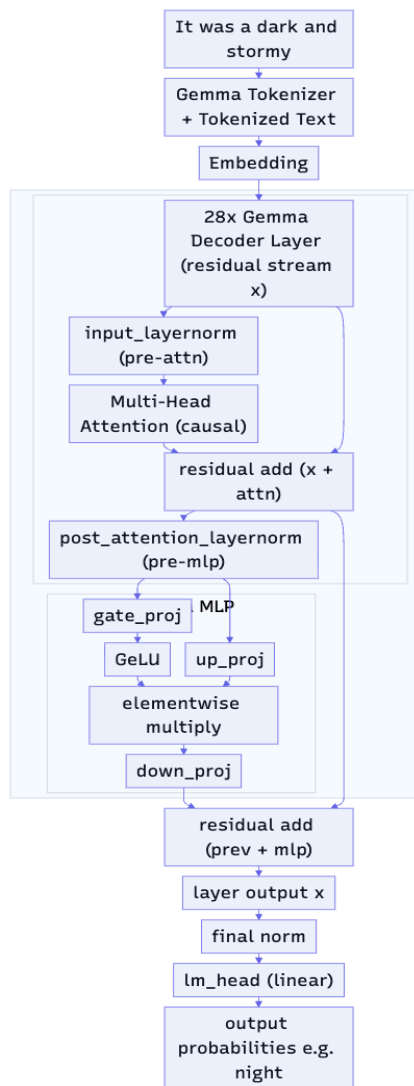
Gemma key model parameters for 2B and 7B models. Adapted from Gemma Team, 2024, p. 2

Parameters	2B	7B
<i>d_model</i>	2048	3072
Layers	18	28
Feedforward hidden dims	32768	49152
Num heads	8	16
Num KV heads	1	16
Head size	256	256
Vocab size	256128	256128

Figure 30 shows the overall architecture of the Gemma 7B model, which was recreated from “Gemma explained: Overview of the Gemma model family architectures,” Google Developers Blog, 2024. The input text is tokenized into embeddings. The input text is tokenized first and then mapped with embeddings. These embeddings are passed through 28 decoder layers, each containing attention and feed-forward blocks. The multi-head attention is used for processing different sequences in parallel with the RoPE positional context. The output passes through GeLU activation before being normalized again, and then a final normalization is applied.

Figure 30

Gemma 7B overall architecture. Recreated from “Gemma explained: Overview of the Gemma model family architectures,” Google Developers Blog, 2024.



Gemma was trained on trillions of web text tokens, and supervised fine-tuning was applied to the Gemma 7B instruct model using a question-and-answer pair dataset, followed by reinforcement learning from human feedback, which helps the model optimize its output for feedback generation. The model comes with an 8 K context window, suitable for handling company metadata and follow-up interview questions, with feedback generation capabilities. The 7B parameter enables the model to produce high-quality output with modest hardware and can be fine-tuned using the LoRA adapters. These properties make this model more aligned for use in implementing this AI mock interviewer system.

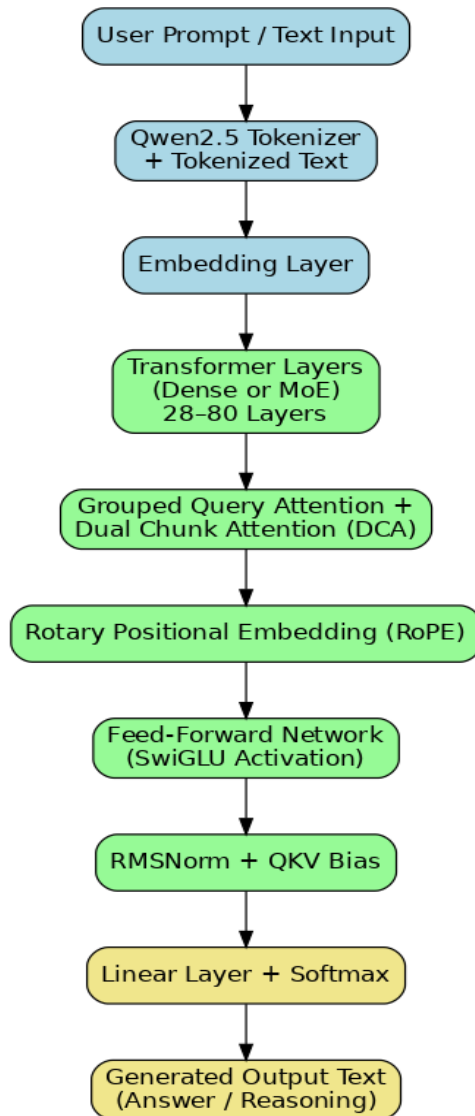
4.1.4 Qwen2.5-7B-Instruct

The Alibaba Research team (Qwen Team et al., 2024) introduced the Qwen 2.5 family model in 2024, which is designed to provide resource-efficient large language models that balance practical computing with strong reasoning capabilities. Qwen2.5-7B-Instruct is the decoder-only transformer model, built on the Qwen and Qwen2 series, and it offers multilingual support with long context handling. The Qwen 2.5 7B instruction is fine-tuned and optimized for instruction-following applications, making it more suitable for interactive systems.

This model incorporates additional refinements in the architecture compared to the original transformer model proposed by Vaswani et al. (2017), including the use of Grouped Query Attention instead of multi-head attention. Additionally, dual chunk attention is used for extending the context length, which is combined with YARN scaling to maintain consistency across longer contexts. Figure 31 illustrates the overall architecture of the Qwen 2.5 7B model, showing the end-to-end flow of input and output. This diagram is recreated based on the information provided by Qwen Team et al. (2024). The user prompt is tokenized first and then embedded before passing through the stacked transformer layers. The model proposes dense or a mixture of experts transformer layers, and either one can be used. The Mixture of Experts layer helps to improve the adaptability over the dense transformation layer. Each layer applies the grouped query attention, dual chunk attention, and SwiGlu activations with RoPE embeddings to refine the representations. Finally, the processed text sequence is sent through a linear layer and softmax to generate the text output.

Figure 31

Qwen 2.5 architecture. Recreated by the author using information from Qwen Team et al. (2024).



The Qwen 2.5 7B instruct model offers a balanced design with a smaller size and higher efficiency and alignment, making it suitable for the AI mock interview system. It can process longer interview sessions with its more extended context capability, and an instruction-tuned model ensures that it provides proper feedback with question-and-answer pairs.

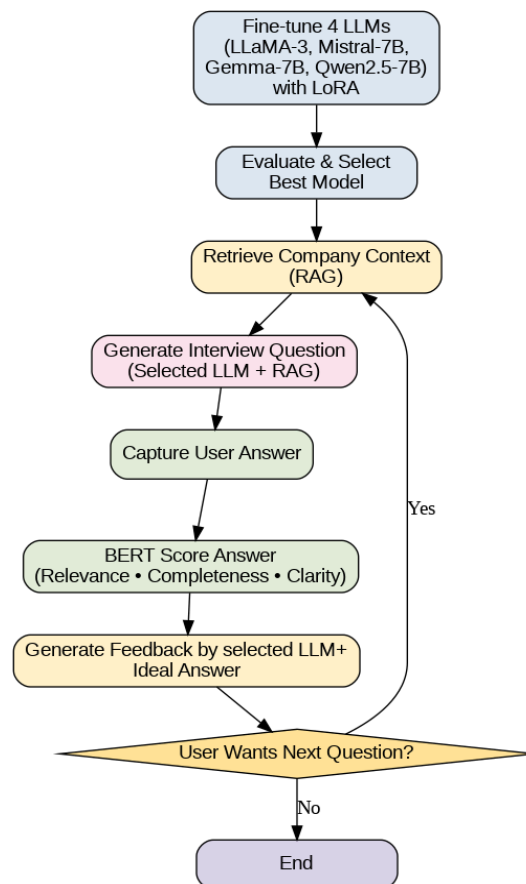
4.1.5 Hybrid model approach

Figure 32 illustrates the innovative hybrid approach, a fine-tuned LLM model enhanced with RAG and BERT scoring to improve company- and role-specific interview questions, along with enhanced

domain-specific feedback and suggested answers. This method uses parameter-efficient fine-tuning LORA for the chosen four models: LLaMA-3, Mistral-7B, Gemma-7B, and Qwen2.5-7B. Evaluating the models using the same metrics, such as perplexity, enables the selection of the best fine-tuned LLM. The Retrieval-based Augmentation is used to improve the fine-tuned LLM prompt with additional company-specific metadata, such as company name, role, and round level, with sample interview questions.

Figure 32

Hybrid approach- LoRA Fine-Tuned LLM with RAG and BERT scoring



Note. Created by Team 3.

A fine-tuned LLM generates the interview question and captures the user's answer. The BERT scoring model is used to assess the user's answer in terms of domain relevance, completeness, and clarity.

All these details, along with the user's answer, are provided to the fine-tuned LLM to receive feedback and suggested answers. If the user wants to continue the interview, the same steps will be repeated. This approach enhances the deployment of a single fine-tuned model into a structured pipeline, thereby improving the AI interview system with maximum adaptability, enhanced domain-specific context, and reasoning, to provide a realistic interview experience to users.

4.1.6 LoRA

The Low-Rank Adaptation (LoRA) is a fine-tuning strategy proposed by Hu et al. (2021) to reduce the training cost of large language models by training only a small subset of parameters. Instead of updating all the weights in the model for fine-tuning, the selected weight layers are inserted with pairs of low-rank matrices. The original model is frozen, and only the assigned weights with new matrices are optimized. This method helps reduce the memory and computational cost required to fine-tune the entire model. Once training is complete, the updated weight adapters are combined with the frozen base model to create a complete fine-tuning model. This method helps to fine-tune the above four chosen models, such as Llama-3-70B-Instruct, Mistral-7B-Instruct-v0.2, Gemma-7B-IT, and Qwen2.5-7B-Instruct, on modest hardware.

4.2 Model Supports

4.2.1 Environment, Platform, Tools

AI Mock Interviewer models employed Google Colab Pro+ and Kaggle Notebooks, which provide access to A100, V100, and T4 GPUs along with high-RAM instances and fast SSD-backed storage. These environments are ideal for model fine-tuning, quantization, embedding generation, and testing multi-model pipelines efficiently.

Our entire application stack is built using Python 3.10+, with PyTorch as the primary deep learning framework. We use the Hugging Face Transformers and PEFT (Parameter-Efficient Fine-Tuning) libraries

to load, quantize, and fine-tune transformer models such as LLaMA 3, Mistral 7B, and Gemini. All fine-tuning is performed using 4-bit quantization with LoRA, targeting only attention projection layers (q_proj, k_proj, v_proj, o_proj) to reduce GPU memory consumption while retaining performance.

For LLaMA 3, we use a 70B quantized variant, optimized through LoRA for question generation and answer feedback tasks. The model is tested locally using Colab, and deployed using Hugging Face Inference Endpoints, enabling scalable and low-latency remote inference for real-time user interactions.

Mistral 7B is used as a lightweight yet powerful alternative, fine-tuned on 1,500 QA-feedback pairs across key domains like Python, SQL, ML, and Behavioral Interviews. This model is well-suited for fast inference, making it a backbone for low-latency question-response cycles. It is trained and tested using the same environment stack as LLaMA 3.

Next model we are choosing to use is Qwen2.5-7B-Instruct and it is a strong choice for building an AI mock interviewer because of its ability to handle very long context (up to 32k tokens). Our next step in project is to customize interview experience based on user resumes and large context size of this model enables us to feed in resumes, job descriptions, and past answers all in one prompt without losing context, this will allow the interviewer to adapt across multiple rounds of questions. Qwen model has strong reasoning skills and it is specifically built for instruction-following makes it effective for both technical and behavioral interview formats. Because of above reasons even in resource constraint environment with efficient fine-tuning via QLoRA on modest hardware, it's scalable for training on custom Q&A data, making it especially useful for realistic, adaptive multi-turn interviews.

Another model that we are choosing is Gemma-7B-IT, it is an excellent option for a AI mock interviewer, as it is already instruction-tuned for conversational tasks, it naturally takes on the role of an interviewer with clear and safe outputs. Its stability and efficiency on smaller GPUs make it accessible for running on Colab. Gemma generates very concise responses, which work well for generating structured feedback after answers. Gemma is best suited for role-play and behavioral interviews where resource efficiency and

reliable interaction quality are more important than extremely long context. All of the chosen models are fully open source and accessible free of cost and best suited for large scale training purposes.

To support Named Entity Recognition (NER), we use Gemini models via API-based inference. Gemini helps extract key metadata like role, company, and interview type from user prompts. Since Gemini provides inference endpoints, we avoid hosting the full model, reducing infrastructure load while ensuring fast response times for entity extraction.

For voice interaction, we integrate OpenAI Whisper, an open-source transformer-based ASR (automatic speech recognition) model. Whisper is used to transcribe spoken answers from candidates into clean, structured text. This module operates as a preprocessor before sending responses to downstream models like LLaMA 3 or Mistral for evaluation. Whisper runs directly within the Colab environment during testing and can be containerized later for deployment.

Table 11

Software and hardware Specifications

Model	Hardware	RAM	SSD
LLaMA 3 (70B)	GPU (NVIDIA A100/T4/V100)	32+ GB	200+ GB
Mistral-7B	GPU (NVIDIA A100/T4/V100)	24+ GB	150+ GB
Gemini (NER API)	API-based (no local compute needed)	N/A	N/A

Qwen2.5-7B-Instruct	GPU (NVIDIA A100/T4/V100)	16+ GB	50+GB
Gema-7B-IT	GPU (NVIDIA A100/T4/V100)	16+GB	40+GB
OpenAI Whisper	GPU/CPU (for inference only)	16+ GB	50+ GB
Hybrid Role Models	GPU (NVIDIA T4/V100)	24+ GB	100+ GB

The specified configurations mentioned in Table 1 ensure compatibility with the latest features and performance optimizations of the respective frameworks, facilitating efficient model training and Deployment.

Table 12
Models Used

Model	Library	Method	Usage
LLaMA 3 (70B)	Hugging Face Transformers, PEFT	LlamaTokenizer, LlamaForCausalLM, get_peft_model	Tokenization, LoRA Fine-Tuning, and Inference
Mistral 7B	Hugging Face Transformers, PEFT	AutoTokenizer, AutoModelForCausalLM, get_peft_model	Tokenization, LoRA Fine-Tuning, and Inference

Gemini (NER)	Gemini API	generateContent, function_calling, NER parsing logic	Named Entity Recognition and Metadata Extraction
OpenAI Whisper	OpenAI Whisper Library	WhisperProcessor, WhisperForConditionalGeneration	Audio Transcription (Speech-to-Text)
Hybrid Role Models	Hugging Face Transformers, PEFT	AutoModel, CustomPromptBuilder, get_peft_model	Role-Specific Model Adaptation and Feedback Logic
Sentence- BERT	sentence-transfor mers	SentenceTransformer, encode	Embedding Generation for Retrieval & Evaluation
RAG Pipeline	Pinecone, ChomaDB	pinecone.init(), faiss.IndexFlatL2, query()	Vector Search and Retrieval-Augmente d Generation

Table 12 illustrates the Model libraries and toolkits used to implement the LLaMA 3, Mistral-7B, Gemini, Whisper, and Hybrid Role-Specific models in our AI Mock Interviewer system. A foundational component of our pipeline is the Hugging Face Transformers library, which provides seamless support for tokenization, fine-tuning, and inference across various large language models. Specifically, it enables us to integrate and customize LLaMA 3 and Mistral with Low-Rank Adaptation (LoRA) for efficient fine-tuning in limited-resource environments.

For Named Entity Recognition (NER), we rely on the Gemini API, which eliminates the need to host heavy models locally while providing fast and accurate metadata extraction from user prompts. This metadata, such as role, company, and interview round, is essential for dynamically generating relevant interview questions.

To support voice-based interaction, we incorporate OpenAI Whisper, an automatic speech recognition model that converts spoken candidate responses into text in real time. This transcription is then passed to downstream components for evaluation, enabling a seamless multimodal user experience.

For semantic search and retrieval, we use Sentence-BERT to generate high-quality embeddings of questions and responses, and store them in Pinecone, a scalable vector database. In some cases, we also use FAISS for fast local retrieval. These tools collectively enhance the RAG (Retrieval-Augmented Generation) capabilities of our system by ensuring that all generated content is contextually relevant to the user’s profile and conversation history.

Overall, the integration of these models and libraries allows the mock interviewer to handle complex NLP workflows—from speech transcription and metadata extraction to dynamic question generation and feedback. This architecture enables adaptive and role-specific interview flows, fostering a more personalized and realistic candidate experience.

Table 13
Libraries and Tools

Libraries/Tools	UseCase
Docker	Containerized environment for model deployment
PEFT	Library to use fine-tuning and LoRA config methods

Transformer	Library to load and work with LLM models and tokenizers
Datasets	Library to load datasets
Pytorch	Pytorch Library to implement architecture, Modify data types
Matplotlib	To create plots of training and validation losses

Jupyter Notebooks and colab notebooks are used to clean and standardize the data. Cleaning and pre-processing involves removing duplicates and unsupported characters, Filtering entries where either questions and answers are empty or in-coherent. After cleaning we augmented data with question categories, levels and company tags wherever possible. Next step was to augment data with feedback and follow up questions. Feedback and followup questions were generated using GPT4 models and then checks the data manually again and preprocesses if the cleaning code does not work in

4.2.2 Model Architecture and Machine Learning Data Flow

Our project involves supervised fine-tuning of large language models (Cheng et al., 2023) to simulate dynamic and adaptive interview experiences. It is this transformer-based architecture, recently described by Jie Cheng et al. (2023) as shown in figure 26 that forms the backbone of our model design and training process. The architecture's core components like self-attention, multi-head attention, and feed-forward layers which enable the models to process candidate responses, interview prompts, and contextual tags with high semantic fidelity and domain-awareness.

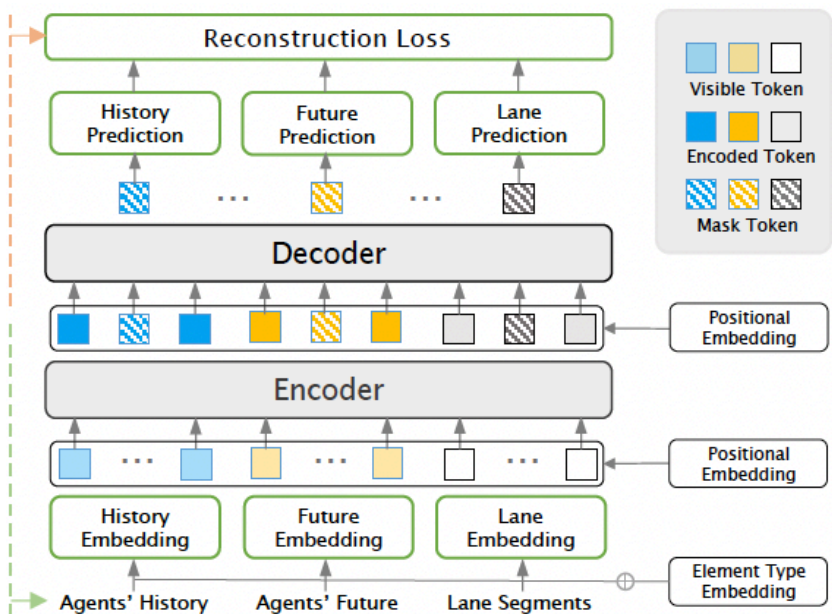
This architecture is leveraged across all key models in our system. LLaMA 3 and Mistral 7B, both transformer-based, are fine-tuned on domain-specific question-answer pairs and used to generate behavioral, technical, and coding questions tailored to the candidate's profile It integrated RAG pipeline

using interview metadata into semantic embeddings, which are then passed to a retriever module backed by a vector database to retrieve top relevant questions for specific company, role and round .

In the generation phase, the decoder architecture employs masked self-attention to selectively attend to relevant context while producing the next question or providing constructive feedback. This fine-tuning strategy enables the models to better capture structured interview flows, handle multi-turn dialogue, and reflect real-world questioning patterns.

Figure 33

Supervised Fine-tuning Architecture described by Jie Cheng et al. (2023)



The AI Mock Interviewer system begins with the user submitting a prompt as shown in figure 27 , which may include their role, company, and interview level. This input is first processed through a Named Entity Recognition (NER) module that combines lightweight models like spaCy with more advanced LLM-based NER using Gemini or LLaMA. The goal is to extract critical metadata—such as job role, domain, and context—which is then used to guide downstream question generation. This modular

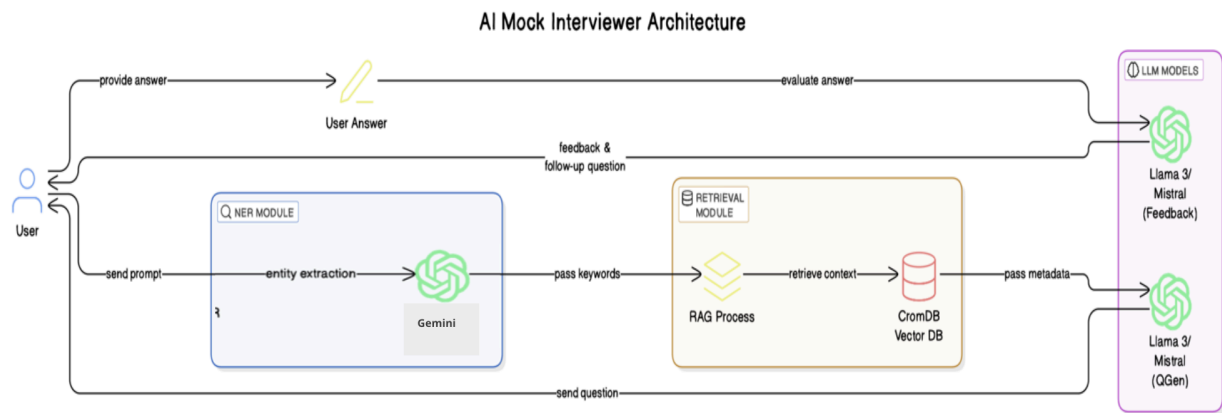
approach allows the system to adapt its behavior to the user's background, ensuring personalized and domain-relevant interview questions.

Next, the extracted entities are passed to a Retrieval-Augmented Generation (RAG) module, which uses the keywords to retrieve relevant context from a ChromaDB vector database. This database stores semantically embedded question-answer pairs tagged with role and company metadata. The RAG component ensures that the system pulls contextually appropriate information, which is then forwarded to the question generation model. Using the retrieved data and metadata, LLaMA 3 or Mistral is called upon to generate a tailored interview question that reflects realistic industry standards and difficulty levels suited to the candidate's profile.

Once the user answers the generated question, the response is sent back into the system, where the same LLM (LLaMA 3 or Mistral) performs answer evaluation and feedback generation. The model analyzes the answer for relevance, clarity, and completeness, and generates constructive feedback, including strengths, areas of improvement, and potential follow-up questions. This loop enables a dynamic and responsive interview simulation experience that mimics real-time interviewer-candidate interaction, with support for multiple rounds and adaptive question flow based on user performance.

Figure 34

Model flow diagram of AI Mock Interviewer



4.3 Model Comparison and Justification

4.3.1 Model Comparison

Table 14

Model Comparison

Model	Targeted Problem	Approach	Key Features	Strengths	Limitations
LLaMA 3	Contextual question generation + feedback generation	Deep Learning (Transformer LLM)	In-context learning, long-form reasoning, prompt adaptation	High fluency, coherent question generation, dynamic adaptation based	Requires large compute; sensitive to prompt phrasing

				on user	
				performance	
Mistral	Question generation and real-time follow-up feedback	Deep Learning (Transformer LLM)	4-bit quantized fine-tuning with LoRA; multi-turn prompt support	Lightweight yet capable; fast inference; deployable on T4/A100; strong instruction tuning	Limited context retention vs larger models; occasional hallucinations in open-domain scenarios
Gemma	Question generation, Real time feed back and followup question	Deep Learning (Transformer LLM)	Fine-tuned with LoRA; Single prompt Completion. Large context window	Lightweight and Open-source. Strong performance per size, Easy Google ecosystem integration	No native chat or multiturn support. Limited capability for instruction tuning
Qwen	Question generation, Real time feed back and followup question	Deep Learning (Transformer LLM)	Fine-tuning 4-bit quantization Instruction	Strong multilingual support. Multi-turn chat support with	Licence restriction for commercial use. Heavier resource requirement.

			tunning. LoRA instruction	Limited	
			adapaters	tunning. ecosystem	
			Competitive		
			performance.		
OpenAI	Voice-to-text	Deep Learning	Multilingual	Robust to noise	Cannot provide
Whisper	transcription	(Sequence-to-Sequence)	STT, prosodic feature extraction, noise tolerance	and accents, multitask trained, integrates easily into real-time systems	feedback by itself; STT only
Hybrid Model	Role-specific customization (DA, DS, DE, MLE)	Deep Learning + Transfer Learning	Fine-tuning per job family, modular weights, LoRA support	High relevance to user context, allows targeted scoring and question control	Needs significant data for each role; model complexity increases with roles

4.3.2 Justification for Model Choices

LLaMA 3 – Interview Question and Feedback Generation

LLaMA 3 was selected for its strong instruction-following capabilities, making it ideal for both generating nuanced, context-sensitive questions and evaluating user responses. Its in-context learning enables real-time adjustment based on prior answers, which is crucial for simulating personalized interview experiences. Its

balance of performance and resource demand (70B version) makes it suitable for both deployment and experimentation.

Mistral-7B – Instruction-Tuned Question and Feedback Generation

Mistral-7B was selected for its efficiency, flexibility, and high-quality output in constrained compute environments. Unlike larger models such as LLaMA 3, Mistral offers a favorable trade-off between generation fluency and computational resource requirements, making it ideal for real-time applications like mock interviews. Its compatibility with 4-bit quantization and LoRA-based fine-tuning significantly reduces memory consumption while preserving performance. Mistral was fine-tuned on 1,500 context-rich QA-feedback samples across multiple interview domains (e.g., SQL, Python, Behavioral), enabling it to generate both interview questions and concise, domain-specific feedback. The model's strong instruction-following capabilities and autoregressive structure make it well-suited for multi-turn interactions and context-aware reasoning. This balance between deployability and generation quality is critical for maintaining interactive speed and accuracy in production-facing AI tools.

Gemma-7B - Instruction Tunned for Question Generation and Answer validation

We selected Gemma-7B model as it is lightweight and open source. These features makes it ideal for private deployment. This model is available in 2B and 7B sizes and with quantization we can use it for efficient inference without requiring massive compute resources. With proper instruction tuning and prompt engineering it can be optimized for language understanding and validating for handling structured interview scenarios, such as behavioural or technical question-answering tasks. Since this model can be deployed locally or on our cloud account, we can maintain full controle over user data. Gemma supports 4-bit quantization along with LoRA and QLoRA fine-tunning which makes it ideal candidate for running on low resource environments. Larger context window of 128K tokens suits well for long interview process for maintaining feedback and generating fallow-up questions.

Qwen-7B -

Qwen comes with instruction fine-tuning and multi-turn conversation capabilities. It is specifically finetuned for dialogue and conversation alignment. It was simulate realistic interview scenario by maintaining context across multiple turns of conversation. It can maintain longer context for asking followup questions and adapt according to user's response. Since qwen has strong multilingual support, in future it opens door for global interview experience. It supports LoRA and QLoRA fine-tuning which allowed us to train model on resource constraint environment.

OpenAI Whisper – Voice-to-Text Pipeline

Whisper's multitask training and robust transcription quality made it a natural choice for handling voice-based inputs. Its ability to accurately convert speech to text even in noisy environments ensures that the user experience is not degraded by technical issues. This model serves as the bridge between the user's spoken input and the text-based evaluation pipeline.

Hybrid Model – Role-Specific Fine-Tuning

To adapt to different job roles, a generic one-size-fits-all model would not suffice. The hybrid model architecture allows us to fine-tune base models specifically for roles like Data Analyst, Data Scientist, Data Engineer, and Machine Learning Engineer. This improves the accuracy and relevance of both generated questions and feedback, ensuring role-aware evaluation.

4.4 Model Evaluation Methods

The AI Mock Interviewer system is evaluated through a range of metrics, each tailored to the specific type of model being tested. For clarity, each metric is described with its formula, the meaning of its variables, its role in evaluation, and its practical relevance.

4.4.1. LLaMA 3 / Mistral 7B / Gemma 7B / Qwen 7B (LLM-based models)

4.4.1.1. Perplexity (PPL)

These large language models are responsible for generating interview questions, interpreting candidate responses, and producing constructive feedback. Their evaluation emphasizes language fluency, semantic accuracy, and factual reliability.

$$PPL = \exp \left(- \frac{1}{N} \sum_{i=1}^N \log P(w_i | w_1, \dots, w_{i-1}) \right)$$

Here, N : Number of tokens in the sequence , $P(w_i | w_1, \dots, w_{i-1})$: Probability assigned to token w_i given previous tokens

Perplexity measures how well the model predicts the next word in a sequence. A lower value means the model is more fluent and coherent. This metric is used during training and validation of LLMs to ensure generated questions and feedback follow natural language flow.

4.4.1.2. BLEU Score

$$BLEU = BP \cdot \exp \left(\sum_{n=1}^N w_n \log p_n \right)$$

In this formula, p_n : Precision of n-grams (fraction of generated n-grams in reference), w_n : Weight for each n-gram , BP : Brevity penalty.

BLEU measures how closely the generated text matches the reference, focusing on phrase overlap. This is valuable because it shows how similar the system’s questions or feedback are to carefully prepared “gold standard” responses. In practice, it is mainly used to benchmark LLM outputs against domain-specific training examples.

4.4.1.3. ROUGE-N

$$ROUGE-N = \frac{\sum_{ref} \sum_{gram_n \in ref} \min(Count_{cand}(gram_n), Count_{ref}(gram_n))}{\sum_{ref} \sum_{gram_n \in ref} Count_{ref}(gram_n)}$$

Here, gram n is an nn-gram sequence, $Count_{cand}(gram\ n)$ is its count in the candidate text, and $Count_{ref}(gramn)$ is its count in the reference.

ROUGE looks at recall, checking how much of the reference text appears in the generated output. It’s especially useful when we need to make sure the system’s feedback is complete and doesn’t miss important technical points.

4.4.1.4. BERTScore (Semantic Similarity)

$$BERTScore = \frac{1}{|X|} \sum_{x \in X} \max_{y \in Y} \cos(e_x, e_y)$$

Where **X**, **Y**: Tokens in candidate and reference texts , **e_x**, **e_y**: Token embeddings , **cos(·)**: Cosine similarity.

Unlike BLEU and ROUGE, which focus on word overlap, BERTScore measures meaning. It checks whether the generated feedback expresses the same intent as the reference. This is valuable in a mock interview setting because the wording can change, but the guidance still needs to be correct and meaningful.

4.4.1.5. Hallucination Rate

$$\text{Hallucination Rate} = \frac{\text{Number of hallucinated outputs}}{\text{Total outputs}}$$

Hallucination rate tracks when the model produces factually wrong or irrelevant responses. It reflects the model's reliability since misleading feedback could harm candidates. The metric is calculated by comparing the number of incorrect outputs against the total responses.

4.4.1.6. Human Evaluation

In addition to automated measures, experts review the generated questions and feedback for relevance, clarity, appropriate difficulty, and overall usefulness. This ensures that the system's outputs are not only accurate on paper but also practical and effective in real interview simulations.

4.4.2. OpenAI Whisper (Speech-to-Text)

Whisper is used to transcribe spoken responses into text. Its evaluation focuses on how accurately it recognizes speech, how well it performs in noisy conditions, and how efficiently it processes audio.

4.4.2.1 Word Error Rate (WER)

$$\text{WER} = \frac{S + D + I}{N}$$

Here, **S**: Substitutions, **D**: Deletions, **I**: Insertions , **N**: Total words in reference.

WER measures transcription accuracy. A lower WER indicates better recognition. It is used when comparing Whisper's transcription against ground-truth text.

4.4.2.2 Character Error Rate (CER)

$$\text{CER} = \frac{S + D + I}{N_{\text{char}}}$$

Where **N_char**: Total characters in reference text.

CER works like WER but at the character level. It is particularly useful for noisy environments and multilingual names or terms.

4.4.2.3 Real-Time Factor (RTF)

$$\text{RTF} = \frac{\text{Processing Time}}{\text{Audio Duration}}$$

If $\text{RTF} < 1$, the transcription is faster than real-time.

This is critical for live interview simulations where delays must be minimal.

4.4.2.4 Noise Robustness

To test noise robustness, Word Error Rate (WER) is calculated under different levels of background noise.

This shows how reliable Whisper is in real-world situations, where candidates may not always have a quiet environment.

4.4.3. Hybrid Role-Specific Models (LoRA / QLoRA fine-tuned)

These models classify answers or provide role-specific validation (e.g., checking if a Data Scientist candidate answered correctly). Evaluation is based on classification metrics.

4.4.3.1 Accuracy

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

Where TP = true positives, TN = true negatives, FP = false positives, FN = false negatives.

Accuracy is a straightforward metric that measures the share of correct predictions overall. It's useful for checking role-based correctness, but on its own it may not tell the full story. That's why we also look at precision, recall, and F1-score.

4.4.3.2 Precision, Recall, F1-Score

$$\text{Precision} = \frac{TP}{TP + FP}, \quad \text{Recall} = \frac{TP}{TP + FN}$$
$$\text{F1} = \frac{2 \times \text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

Precision shows how many predicted positives are actually correct, recall shows how many of the real positives were identified, and F1 provides a balance between the two. These metrics are especially important when the data is imbalanced, where accuracy alone could be misleading.

4.4.3.3 Confusion Matrix

$$CM = \begin{bmatrix} TP & FP \\ FN & TN \end{bmatrix}$$

A confusion matrix shows exactly where the model is making mistakes. It is applied when analyzing misclassifications, such as confusing a Data Analyst response for a Data Scientist one.

4.4.3.4 ROC and AUC

$$TPR = \frac{TP}{TP + FN}, \quad FPR = \frac{FP}{FP + TN}$$

TPR is the True Positive Rate and FPR is the False Positive Rate.

The ROC curve plots TPR vs FPR, and the AUC is the area under this curve. AUC close to 1 indicates a strong classifier. This metric matters for checking consistency across thresholds.

4.4.3.5 Cross-Entropy Loss

$$L_{CE} = - \sum_{i=1}^N y_i \log \hat{y}_i$$

Where y_i : True label, $\hat{y}_i$: Predicted probability.

This is the training loss function. It is used during fine-tuning to monitor learning stability and convergence.

4.4.4. System-Level Evaluation (End-to-End Interviewer)

At the system level, evaluation goes beyond individual models to confirm that the entire pipeline provides a smooth and useful interview experience:

4.4.4.1 Dialogue Success Rate

$$\text{Dialogue Success Rate} = \frac{\text{Successful sessions}}{\text{Total sessions}}$$

This measures how many interviews run successfully from start to finish without issues. It reflects the system's ability to maintain a coherent conversation.

4.4.4.2 User Satisfaction Score

Collected through surveys (e.g., Likert scale or ratings), this captures how realistic, challenging, and helpful users find the mock interviews. User trust and adoption depend heavily on these perceptions.

4.4.4.3 Latency

$$\text{Latency} = \frac{\text{Total Response Time}}{\text{Number of interactions}}$$

Latency is the average response time between a user's input and the system's reply. Lower latency makes the interaction feel more natural and conversational.

4.4.4.4 Throughput

$$\text{Throughput} = \frac{\text{Requests processed}}{\text{Time elapsed}}$$

Throughput measures how many interview interactions the system can handle within a given period. This is key for scaling the platform to support many users at once.

4.4.4.5 A/B Testing

Two different models (such as LLaMA and Mistral) are tested with real users, and metrics like BLEU, BERTScore, latency, and satisfaction are compared. This helps identify which model performs best for deployment.

5. Data Analytics System

5.1 System Requirements Analysis

5.1.1.1 System Boundary

Inside the System Boundary:

- React/Next.js frontend, FastAPI backend on Cloud Run
- Four fine-tuned LLMs (LLaMA-3-70B, Mistral-7B, Gemma-7B, Qwen2.5-7B)
- RAG pipeline with ChromaDB vector database
- Gemini NER module for metadata extraction
- OpenAI Whisper for speech-to-text
- BERT scoring engine for answer evaluation
- BigQuery data warehouse and Google Cloud Storage

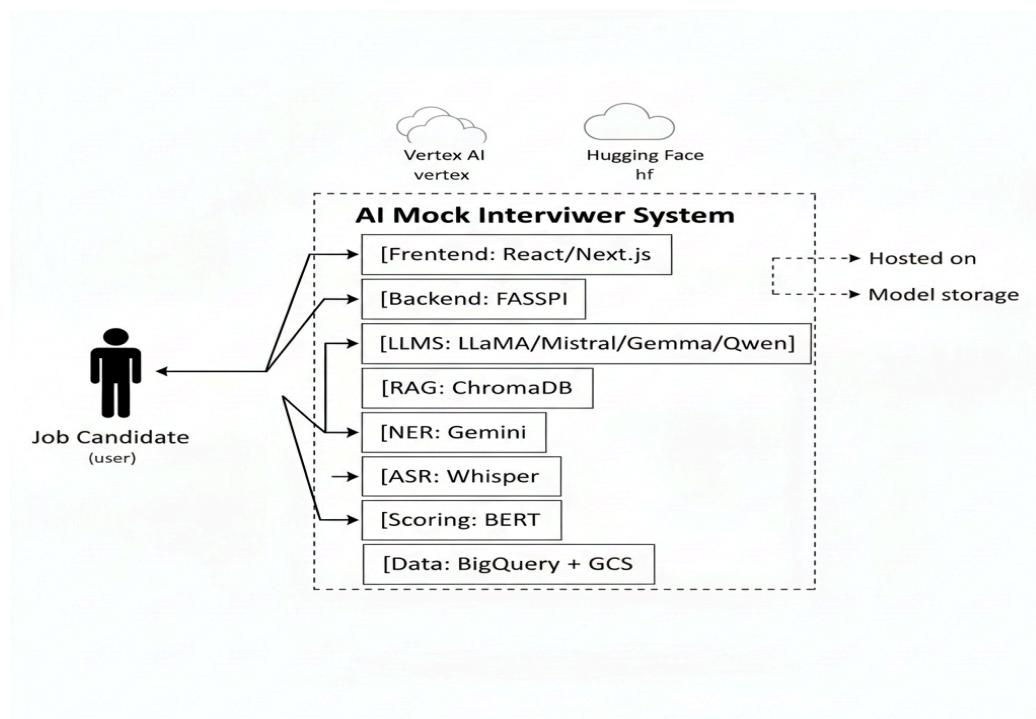
- Cloud Functions for automated data processing
- Vertex AI for model hosting and embeddings

Outside the System Boundary:

- Job candidates (end users)
- Hugging Face Hub and Vertex AI (external services)
- Training environments (Colab, Kaggle)
- User devices and browsers

Figure 35

System Boundary Diagram showing internal components and external entities



5.1.1.2 System Actors

Primary Actors:

1. **Job Candidate:** Configures interview sessions, provides answers via text/voice, receives questions and feedback, views performance analytics
2. **System Administrator:** Manages deployments, uploads datasets, monitors system health, updates models

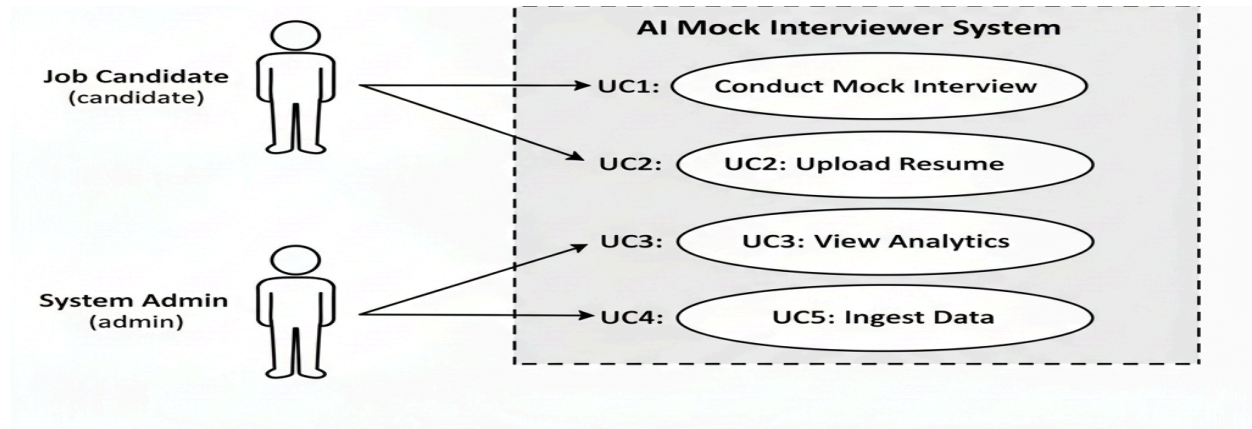
Secondary Actors:

3. **Data Pipeline (Automated):** Converts Excel to CSV, validates schemas, generates embeddings, synchronizes ChromaDB
4. **Training Pipeline (Automated):** Fine-tunes models using LoRA, evaluates performance, deploys to production

5.1.1.3 Use Cases

Figure 36

Use Case 1: Conduct Mock Interview Session



Actor: Job Candidate

Precondition: User authenticated and in interview lobby

Flow: User provides company/role/level → Gemini NER extracts metadata → RAG retrieves relevant questions from ChromaDB → LLM generates personalized question → User answers (text/voice) → Whisper transcribes if voice → BERT scores answer quality → LLM generates feedback and follow-up → Process repeats for multiple rounds → Final performance report generated

Postcondition: User receives comprehensive feedback with competency scores and improvement suggestions

Use Case 2: Upload and Analyze Resume

Actor: Job Candidate

Precondition: User has resume file (PDF/DOCX)

Flow: User uploads resume → System extracts text → Gemini NER identifies skills and experience → Resume scored for ATS compatibility and keyword density → Interview questions customized based on resume keywords

Postcondition: Personalized interview aligned with candidate's background

Use Case 3: Data Ingestion and Embedding Generation

Actor: System Administrator

Precondition: New question dataset available in Excel format

Flow: Admin uploads file to GCS → Cloud Function converts to CSV and validates schema → BigQuery queries data → BigQuery ML generates embeddings via Vertex AI → Embeddings stored and synchronized with ChromaDB

Postcondition: New questions available for RAG retrieval in production

Use Case 4: Train and Deploy Fine-Tuned Model

Actor: System Administrator

Precondition: Training dataset with 1,000+ QA examples

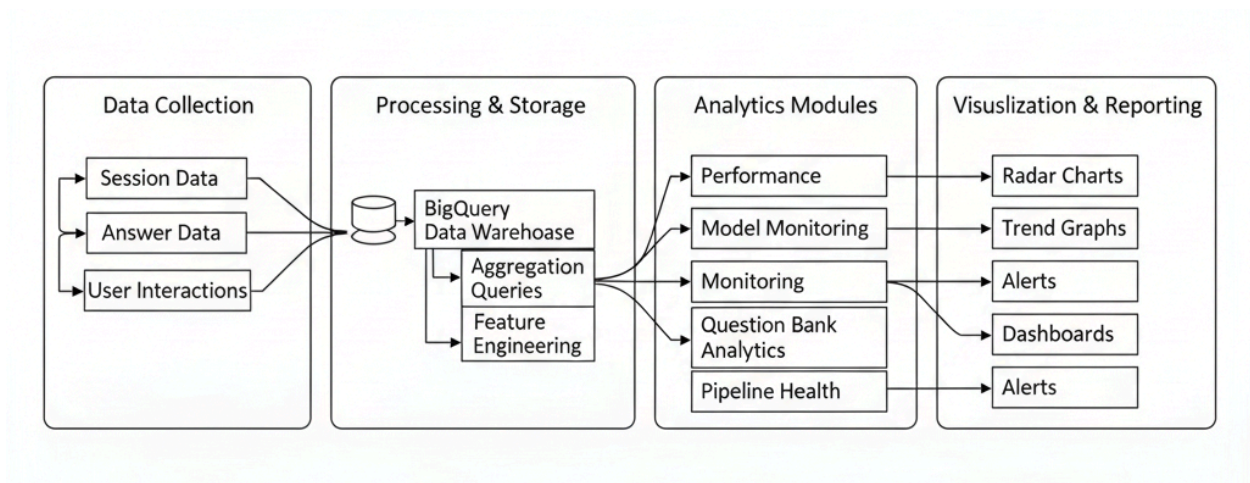
Flow: Dataset loaded in Colab/Kaggle → Model fine-tuned using LoRA with 4-bit quantization → Evaluated using perplexity, BLEU, BERTScore → Adapter uploaded to Hugging Face Hub → Deployed to Vertex AI endpoint → Backend routes traffic to new model

Postcondition: Updated model serving production traffic with improved quality

5.1.2 High-Level Data Analytics and Machine Learning Functions

Figure 36

Analytics Pipeline Architecture



5.1.2.1 Data Analytics Functions

1. Interview Performance Analytics

- Calculates domain-specific scores (Python, SQL, ML, Behavioral)
- Tracks time-per-question and engagement metrics
- Generates radar charts for competency visualization
- Compares performance across multiple sessions to track improvement
- Provides percentile rankings against peer benchmarks
- All data stored in BigQuery with indexes on user_id and session_date

2. Model Performance Monitoring

- Tracks quality metrics: perplexity, BLEU, ROUGE, BERTScore
- Monitors hallucination rates (target <2%)
- Measures inference latency (P50, P95, P99) and throughput
- Collects user satisfaction via post-interview surveys
- Implements A/B testing to compare model versions
- Real-time dashboards via Looker Studio with alerting

3. Question Bank Analytics

- Identifies frequently/rarely retrieved questions
- Analyzes embedding space for topic coverage gaps
- Tracks user success rates to calibrate difficulty levels
- Monitors metadata completeness and data quality

4. Data Pipeline Health Monitoring

- Tracks ingestion success rates and schema validation
- Monitors embedding generation performance and API quotas
- Verifies ChromaDB synchronization status
- Projects storage growth for capacity planning

5.1.2.2 Machine Learning Functions and Capabilities

1. Question Generation via Fine-Tuned LLMs

Models: LLaMA-3-70B (70B params, 8K context), Mistral-7B (sliding window attention, 4K context), Gemma-7B (28 layers, 8K context), Qwen2.5-7B (32K context with dual chunk attention)

Training: LoRA fine-tuning with 4-bit quantization on Colab A100 GPUs; Mistral trained on 1,500 examples, Gemma on 28,000+ examples; targets attention layers (q_proj, k_proj, v_proj, o_proj)

Capabilities:

- Generate domain-specific questions (Python, SQL, ML, Behavioral)
- Adapt to company culture and interview round
- Maintain multi-turn conversation context
- Create follow-ups based on previous answers
- Deployed on Vertex AI with 1.2-2.5s latency

Evaluation: Gemma achieved perplexity 12.37, BLEU 0.471, validation loss 1.9254

2. Answer Evaluation and Scoring

Model: Sentence-BERT generates 768-dimensional embeddings for semantic similarity

Scoring Dimensions:

- Domain Relevance (40%): Semantic alignment with question topic
- Completeness (40%): Coverage of expected answer components
- Clarity (20%): Coherence and logical structure

Output: Numerical scores (0-100) stored in BigQuery with confidence intervals

3. Feedback Generation

Approach: Same fine-tuned LLMs generate constructive feedback based on answer text and BERT scores

Components:

- Identifies strengths in candidate's response
- Highlights improvement areas with specific examples
- Provides comprehensive model answer
- Suggests actionable study resources

Process: Answer + scores combined in structured prompt → LLM generates 100-200 word feedback → Post-processed for quality and formatting

4. Named Entity Recognition (NER)

Model: Gemini API with function calling

Extracts:

- Company name with standardized mapping
- Job role (Data Scientist, ML Engineer, etc.)

- Interview level (Round 1, 2, 3)
- Skills and keywords from resume text

Implementation: REST API with JSON output validation against known entity lists

5. Retrieval-Augmented Generation (RAG)

Architecture:

- Vertex AI textembedding-gecko generates 768-dim embeddings
- ChromaDB stores embeddings with metadata (company, role, level, tags)
- HNSW index enables sub-100ms similarity search
- Cosine similarity retrieval with metadata filtering

Process: User metadata → Query embedding → ChromaDB search → Top-k results filtered by company/role/level → Questions passed to LLM

Data Flow: Excel upload → Cloud Function → CSV → BigQuery → Embedding generation → ChromaDB sync → Production retrieval

6. Speech-to-Text Transcription

Model: OpenAI Whisper Large-v3 (1.55B params, encoder-decoder transformer)

Capabilities:

- 95% accuracy across diverse accents
- Noise robust (SNR as low as 10 dB)
- Real-time performance (RTF < 1)
- Word-level timestamps for synchronized display

Metrics: Target WER <5% for clear audio, <10% for noisy environments

Deployment: Containerized on Cloud Run; audio uploaded to GCS, URLs sent to endpoint, transcription returned as JSON

7. Resume Analysis

Pipeline: Text extraction → Section segmentation → NER entity extraction → TF-IDF keyword extraction

Scoring (0-100):

- ATS Compatibility (30%): Standard headers, readable format
- Keyword Density (40%): Match with target role requirements
- Formatting Quality (20%): Structure, length, hierarchy
- Impact Language (10%): Quantifiable achievements

Output: Overall score with detailed breakdown, improvement suggestions, and question personalization weights

8. Model Training and Optimization

Infrastructure: Google Colab Pro+ (A100 40GB), Kaggle (T4/P100); PyTorch 2.1+, Transformers 4.36+, PEFT, bitsandbytes

LoRA Config: Rank 16-32, alpha 32-64, 0.05 dropout; targets attention layers; trains ~1% of parameters

Quantization: 4-bit NF4 reduces memory 70% with <2% quality loss

Hyperparameters: Learning rate 2e-4, batch size 4, gradient accumulation 4, 3 epochs, AdamW optimizer, bfloat16 precision

Metrics: Perplexity <15, BLEU >0.4, ROUGE-L >0.5, BERTScore F1 >0.85, Self-BLEU <0.05

9. Long-Context Processing

Model: Qwen2.5-7B with 32K token context window

Architecture: Dual chunk attention + YARN scaling for extended sequences

Use Cases:

- Full resume + conversation history + retrieved context (10K+ tokens)
- Multi-document reasoning across job descriptions and company info
- Code review scenarios with complete submissions

Performance: Stable perplexity across 32K tokens; 40% latency increase vs 4K baseline

10. Hybrid Multi-Model Ensemble

Strategy:

- LLaMA-3-70B: Complex reasoning, experienced candidates
- Mistral-7B: Fast inference, 60-70% of traffic, junior candidates
- Gemma-7B: Safety-critical feedback generation
- Qwen-7B: Long-context resume-integrated interviews

Benefits: 60% cost reduction, improved latency (1.2s vs 2.5s), task-optimized quality, automatic failover

5.2 System Design

5.2.1 System Architecture and Infrastructure

The AI Mock Interviewer system follows a modular, cloud-native architecture that ensures scalability, fault tolerance, and ease of maintenance. At the highest level, the system consists of a web-based frontend, a containerized backend API, AI-powered services for question generation and semantic retrieval, and a robust data layer for storage and analytics.

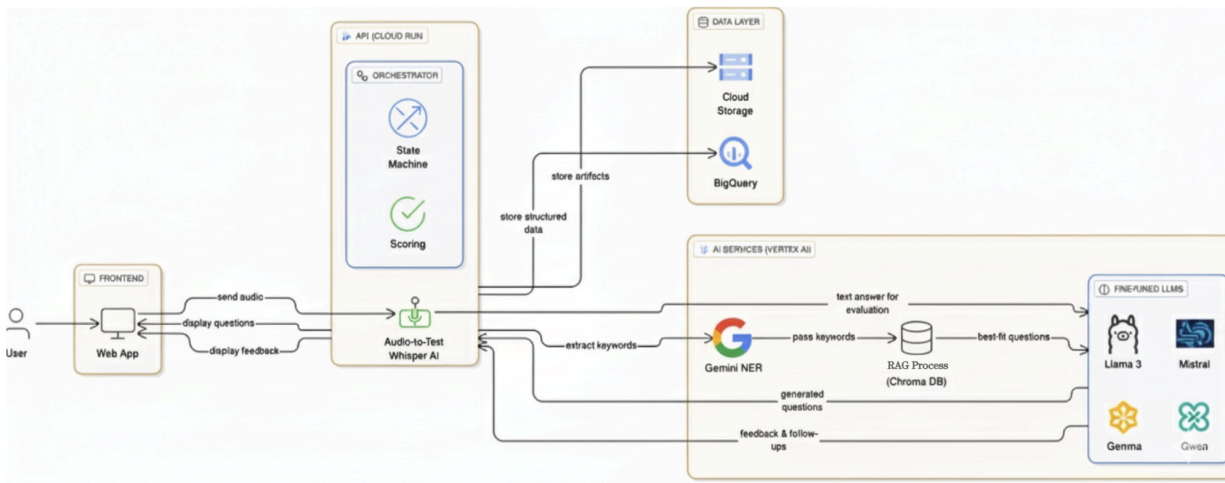
The frontend is developed using React and Next.js, enabling a fast and interactive user experience. It provides all candidate-facing features including the interview lobby, live question display, answer input, hints and feedback, and the final results dashboard. The backend is implemented using FastAPI or Node.js and is deployed on Google Cloud Run for auto-scaling and easy version management. This backend handles session management, communicates with the conversation orchestrator, retrieves relevant content, and coordinates with AI services.

The conversation orchestrator is responsible for maintaining the interview state machine, generating contextually relevant prompts, and invoking scoring and evaluation routines. AI capabilities are powered by 4 fine-tuned models; Llama 3, Mistral, Gemma and Qwen via Vertex AI, which are used to generate and deliver feedback aligned with rubric-based scoring. Named entity recognition is implemented with Gemini and is used to extract the keywords like company, role and level from the user prompt. This data is passed to the RAG pipeline.

The RAG (vector search) layer is implemented with ChromaDB to perform low-latency semantic retrieval of relevant questions for the particular role, company and level. Whisper AI is used for speech-to-text service that transcribes the audio to text, then forwarded to the orchestrator and passed to the LLM during the interview. BigQuery acts as the primary data warehouse for storing structured interview data, logs, and analytics, while Google Cloud Storage (GCS) holds raw and curated data artifacts.

Figure 37

Data Pipeline



5.2.2 System supporting platforms, frameworks and a cloud environment

Google Cloud Platform (GCP) serves as the primary cloud environment, forming the foundation for deploying the AI Mock Interview platform in a scalable and cost-efficient manner. Containerized services run on Cloud Run, allowing automatic scaling and smooth version rollouts. Google Cloud Functions form a serverless, event-driven data ingestion pipeline that converts Excel files to CSV format immediately after upload. Google Cloud Storage (GCS) is used for staging data across landing and processed data buckets.

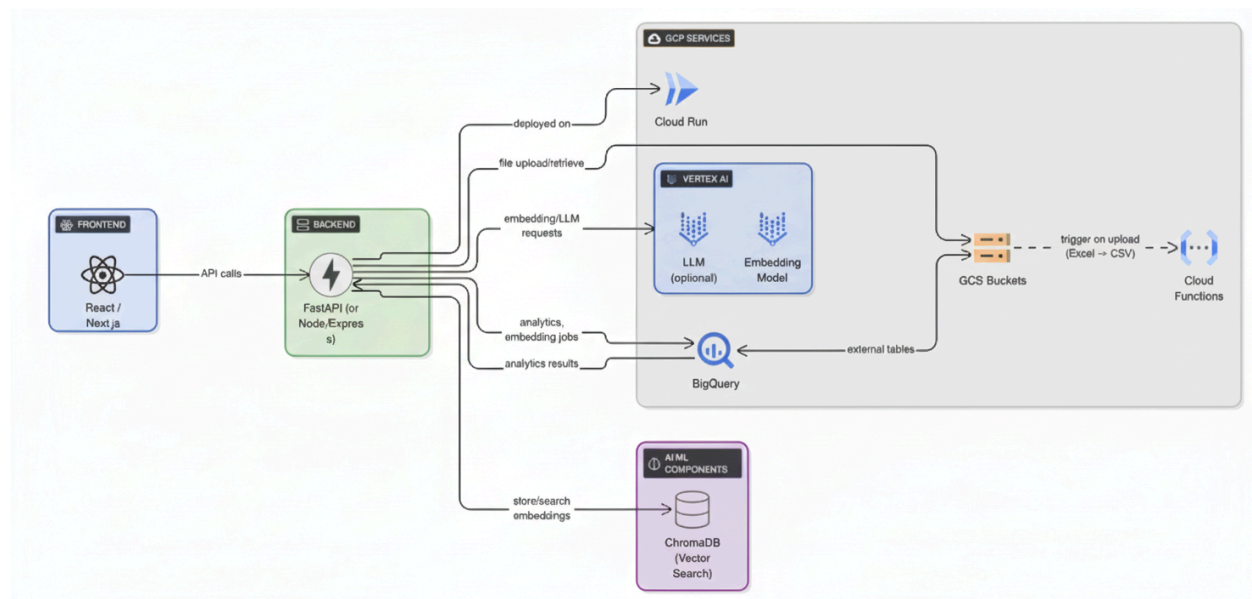
BigQuery acts as the data warehouse, providing external tables for curated data and enabling SQL-based feature generation and embedding jobs. Vertex AI is integrated to generate text embeddings through the textembedding-gecko model, powering the semantic search capabilities of the system.

Postman is used during development to test APIs and ensure seamless integration between components, while Google Colab is utilised for early experimentation, API calls, and exploratory data analysis.

The system also leverages the Hugging Face ecosystem for model experimentation and fine-tuning. Datasets prepared in BigQuery can be exported for training and model management. Python with FastAPI powers the backend APIs, while React and Next.js deliver a rich and responsive frontend experience.

Figure 38

System flow diagram



5.2.3 System data management solution and database

The data management solution focuses on automated ingestion, transformation, storage, and retrieval. The process begins when users upload Excel files containing interview questions or configuration data. These files are stored in the landing bucket on GCS. A Cloud Function is triggered

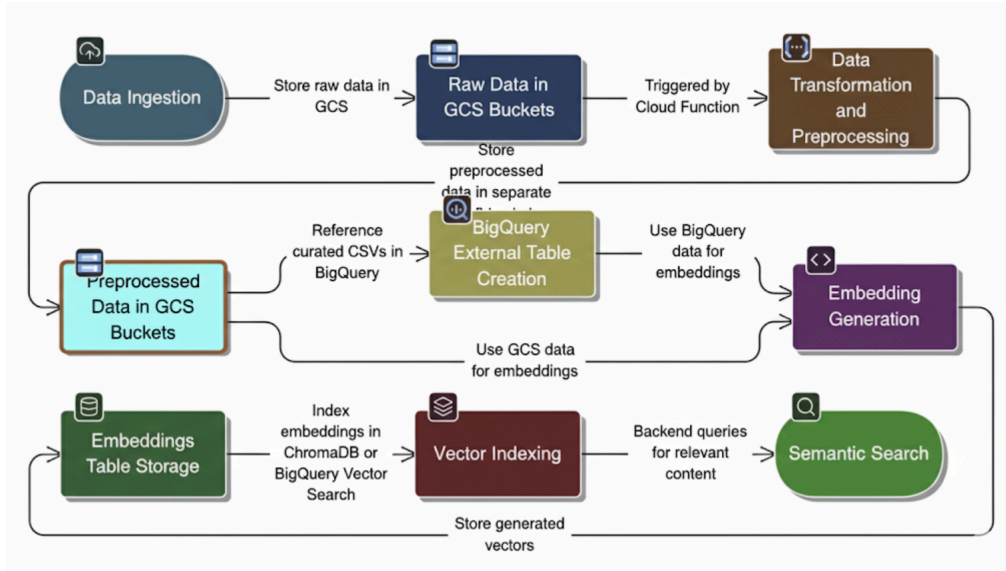
immediately to convert the Excel files into CSV format, run schema validation, and move the validated data into the curated bucket.

BigQuery uses external tables to query curated data without duplicating it. After the data is available in BigQuery, relevant text fields such as questions, categories, and descriptions are concatenated using SQL transformations. Embedding generation jobs are executed by BigQuery ML connected to Vertex AI, producing high-dimensional vector embeddings that represent semantic meaning. The resulting embeddings are stored in a dedicated embeddings table and are periodically refreshed to keep up with new data uploads. These embeddings are then synchronized with ChromaDB, where a vector index is built to support efficient similarity search. The backend queries this index at runtime to retrieve contextually relevant data that guides interview question generation and feedback. This automated data flow ensures that the system always operates on the most current and accurate dataset.

The database schema is carefully designed to capture all key aspects of the interview process. It includes tables for users, sessions, interview user data such as role, company, and level, conversation turns capturing both questions and answers, feedback for evaluation, and content items that populate the question bank. Each table includes metadata fields for traceability, timestamps to track changes, and model versioning to ensure reproducibility of results. Indexes are created on high-usage fields like session date and user ID to optimize query performance

Figure 39

Data Management



5.2.4 System user interface and data visualization

The user interface is designed to be intuitive, interactive, and accessible. Candidates begin in a lobby where they select the role, company and level. Also, the candidate can upload their resume and the system scans it and generates a score. During the interview session, the interface displays dynamically generated questions one at a time, capturing candidate responses through text or audio. Audio answers are processed and transcripts are displayed in real time. Immediate feedback cues and coaching hints can be presented after each answer, guiding candidates toward structured responses. Visual indicators such as timers, progress bars, and question counters keep candidates informed of their progress and remaining time, reducing anxiety and creating a more realistic simulation of a live interview.

After the interview concludes, the results dashboard presents a detailed performance summary. Competency scores are visualized using radar charts for easy comparison across dimensions like communication, problem solving, and technical knowledge. Bar charts and line graphs illustrate score trends, time spent per question, and token usage during responses. A narrative summary highlights key strengths and areas needing improvement, enabling candidates to focus their preparation effectively.

5.3 Intelligent Solution

5.3.1 AI and Machine Learning Model Development

The AI Mock Interviewer system leverages a series of domain-adapted, fine-tuned large language models (LLMs) to generate personalized, context-aware interview experiences. The current system integrates multiple transformer-based models including Mistral-7B-Instruct, LLaMA-3-8B-Instruct, and Gemma-7B-IT, with planned support for Qwen2.5-7B-Instruct in the next iteration. These models are fine-tuned using Low-Rank Adaptation (LoRA) and 4-bit quantization techniques to optimize memory usage and training efficiency while maintaining high output quality.

For Mistral and LLaMA 3, instruction-based fine-tuning was conducted using a curated dataset of over 1,500 domain-specific question–answer–feedback samples across multiple interview categories such as Python, SQL, Machine Learning, Data Analytics, and Behavioral Interviews. The models were adapted to follow a structured prompt format and generate both realistic interview questions and constructive feedback based on user answers.

The Gemma-7B-IT model is currently being fine-tuned on a significantly larger dataset of over 28,000 examples. This dataset includes detailed interview dialogues with labeled domains and follow-up prompts. Using a 3-epoch training regime on Colab A100 environments with LoRA adapters targeting key transformer layers (e.g., `q_proj`, `v_proj`, `gate_proj`), the Gemma model achieved strong convergence with a best validation loss of 1.9254. Evaluation metrics indicate promising performance with a BLEU score of 0.471, Self-BLEU of 0.026, and perplexity of 12.37, affirming its fluency and generation stability.

This modular approach allows different models to be deployed based on task specificity and hardware constraints. For example, Mistral-7B is lightweight and suitable for low-latency feedback generation, while LLaMA-3 handles multi-turn reasoning with high contextual fidelity. Gemma-7B, on

the other hand, is optimized for conversational fluency and safe generation. The Qwen2.5-7B-Instruct model, currently under integration, will further enhance support for long-context interactions, enabling the system to incorporate user resumes, prior responses, and company profiles all within a single prompt.

The overall system supports iterative re-training using newly collected user interactions. This design ensures continuous improvement of model outputs based on real-world usage patterns, helping the AI interviewer evolve in both domain understanding and response quality.

Table 15

Model Fine-tuning parameters and experimental setup

Model	Base Model / Epoch/Steps	Batching / Seq. Length	Optimized Learning Rate	LoRA Setup
Gemma-7b-it	Google’s Gemma-7b-it / 3 epochs	Effective batch size = 24 / 2048 tokens	1e-4 (cosine decay + 10% warmup)	Rank = 16, Alpha = 32, Dropout = 0.1 — captures new patterns without heavy compute, scaling = 32
Mistral-7B-Instruct	Mistral-7B-Instruct-v0.2 / 2000 steps	Effective batch size = 16 (2 per	2e-4 (linear schedule + 100 warmup steps)	Rank = 16, Alpha = 32, Dropout = 0.05 — LoRA

		device $\times$ 8 acc.) /		on q_proj, k_proj, v_proj,
		2048 tokens		o_proj; scaling = 32
LLaMA-3-8	Meta-LLaMA-3	Effective batch	2e-4 (linear	Rank = 16, Alpha = 32,
B-Instruct	-8B-Instruct /	size = 16 (2 per	schedule + 100	Dropout = 0.05 — LoRA
	2000 steps	device $\times$ 8 acc.) /	warmup steps)	on q_proj, k_proj, v_proj,
		2048 tokens		o_proj; scaling = 32

5.3.2 Implemented Design System

The backend of the AI Mock Interview platform is built around parameter-efficient fine-tuning methods and resource-conscious model design. All models undergo LoRA-based fine-tuning with 4-bit quantization, drastically reducing GPU memory consumption—by as much as 65–70%—without sacrificing generation quality. Fine-tuning is performed on consumer-accessible hardware such as NVIDIA T4 and A100 GPUs through platforms like Google Colab Pro+.

Training is conducted using the Hugging Face Transformers and PEFT libraries, enabling modular configuration of learning rates, gradient accumulation, and mixed-precision (FP16 or bfloat16) training. Each model is fine-tuned to handle multi-turn question-answer-feedback flows and generate follow-ups conditioned on user performance. Specific adapter layers targeted include the attention projection heads (q_proj, k_proj, v_proj, o_proj) as well as feed-forward layers (gate_proj, up_proj, down_proj) in Gemma’s case.

The models are saved and deployed as lightweight adapters, reducing storage size and enabling quick switching between model variants. Model weights are stored on Hugging Face Hub with version

control, supporting both local and remote inference pipelines. Whisper is used for real-time voice transcription, while question generation and feedback evaluation are triggered via API endpoints within a Flask or FastAPI backend.

Our AI Mock Interviewer is powered by a Retrieval-Augmented Generation (RAG) pipeline that combines a structured workbook, ChromaDB, and Gemini to create realistic, company- and role-specific interview practice.

The process starts with the dataset, which acts as the ground truth. It contains the company, role, interview round details, tags, and a bank of curated interview-style questions. This data is cleaned, structured, and converted into embeddings, which are then stored in ChromaDB along with metadata. Because each question is tagged with round type and skill area, the system can always pull back questions that are both accurate and relevant.

When a candidate enters a query like *“Help me practice for a Data Scientist Round 3 interview at Meta”*, the system first extracts the key details (company, role, round). It then searches ChromaDB for the best-matching questions. Those questions are passed through Gemini, which makes them sound more natural and conversational like how a real interviewer would phrase them.

The interview experience is designed to feel interactive. A retrieved question is asked, the candidate answers, and the model provides feedback along with a follow-up question. This loop creates an adaptive dialogue that closely mirrors the flow of an actual interview, helping candidates practice not just answering, but also handling feedback and follow-ups.

This setup offers several benefits:

Personalization: Metadata ensures the questions are tailored to a specific company, role, round, or even focus area like SQL, behavioral, or product cases.

Consistency with adaptability: ChromaDB grounds every session in curated, reliable data, while Gemini adapts the phrasing and follow-ups to keep the flow natural.

Scalability: New companies, roles, or interview rounds can be added simply by updating the workbook, making the system easy to grow.

Real-world simulation: The feedback loop teaches candidates to refine their answers, think on their feet, and respond to dynamic interviewer prompts.

Together, this hybrid approach ensures that interview preparation is not only relevant and accurate but also engaging, conversational, and realistic.

5.3.3 Input, Output, and Supporting System Context

The system is designed to accept structured prompts as input and return contextually aligned questions and feedback. The input schema typically includes a domain, company, and optional user role metadata. For example:

Input Text: Generate a question for Data Analytics at Nvidia

Domain: Data Analytics at Nvidia

Results in:

Generated Question: What is a z-score?

This input-to-output generation is not static—it varies dynamically depending on domain (e.g., Machine Learning at AWS vs. Data Science at SAP), company name, or candidate profile. Figure below shows real examples:

Input Text: Generate a question for Data Science at SAP

Generated Question: Can you tell me about a time when you had to make a decision based on a hypothesis?

Input Text: Generate a question for Machine Learning at AWS

Generated Question: What is the difference between supervised and unsupervised learning?

The output continues through the answer-evaluation loop. After a candidate response, the model provides real-time feedback and generates a new follow-up, adapting the difficulty and relevance as the session progresses. This approach ensures that each user receives a tailored mock interview experience.

To support these operations, the backend is equipped with multiple services:

- Whisper API for converting user voice input into text.
- LLM inference endpoints (Hugging Face or Google Cloud Vertex AI) for executing question and feedback generation.
- Retrieval-Augmented Generation (RAG) support using Sentence-BERT and ChromaDB for semantic search and context injection.
- Flask/FastAPI-based custom endpoints to handle `generate_question`, `submit_answer`, and `get_feedback` actions.

Google Cloud Storage and BigQuery are used for structured data logging, analytics, and evaluation.

This integrated design supports not only dynamic question generation but also multi-turn adaptability, accurate assessment, and real-time feedback—creating a highly interactive and intelligent mock interview experience for data professionals.

5.4 System Supporting Environment

The AI Mock Interviewer system is built on a modular, scalable, and cloud-native architecture that integrates industry-standard technologies, platforms, and frameworks to ensure smooth development, training, deployment, and real-time interaction.

5.4.1 Cloud Infrastructure and Hosting Platforms

The project leverages Google Cloud Platform (GCP) as its core infrastructure. Backend services are containerized and deployed via Cloud Run, enabling autoscaling and seamless version management. Real-time inference and semantic search are powered through Vertex AI, which hosts and serves fine-tuned large language models such as LLaMA 3, Mistral, and Gemma. The system also uses Google Cloud Functions for serverless, event-driven processing, particularly for data ingestion and file transformation tasks.

Google Cloud Storage (GCS) is used for storing datasets, interview logs, resume files, and preprocessed embeddings, while BigQuery acts as the primary data warehouse for structured interview data, feature generation, and analytics. The cloud-first approach ensures high availability, fault tolerance, and the ability to scale with user demand.

5.4.2 Model Training and Experimentation Environment

Training and evaluation of models are performed on Google Colab Pro+ and Kaggle Notebooks, which provide access to high-performance NVIDIA A100, T4, and V100 GPUs. These platforms allow fast prototyping, fine-tuning, and evaluation of transformer models with 4-bit quantized LoRA (Low-Rank Adaptation) configurations.

Model development utilizes the PyTorch framework, with support from the Transformers, PEFT, and Accelerate libraries from Hugging Face for fine-tuning and efficient inference. LoRA adapters are

trained and stored for models such as Mistral-7B-Instruct, Gemma-7B-IT, and Meta-LLaMA-3-8B-Instruct, allowing reusable and memory-efficient training.

Trained models and adapters are hosted on Hugging Face Hub, which supports both version control and public sharing. Additionally, Hugging Face Inference Endpoints are used to expose REST APIs for real-time question generation and feedback prediction.

5.4.3 Programming and Development Tools

The primary development language is Python 3.10+, and model implementation, data processing, and evaluation are performed using Jupyter Notebooks and Visual Studio Code (VSCode). Python packages such as datasets, tqdm, matplotlib, sentence-transformers, and evaluate are used for training, visualization, and performance benchmarking.

The system includes support for voice-based interaction through OpenAI Whisper, which performs automatic speech recognition (ASR) to transcribe candidate audio responses. These transcripts are then analyzed by the LLMs to generate structured feedback.

Sentence-BERT is used for embedding generation to support the RAG (Retrieval-Augmented Generation) pipeline, which retrieves semantically relevant context for interview sessions from ChromaDB and FAISS-backed vector stores.

5.4.4 Project Collaboration, Visualization, and Monitoring

GitHub is used as the central repository for all code, scripts, model checkpoints, and logs. Each model version and experimental configuration is tracked for reproducibility. Monday.com serves as the task management platform for project planning, milestone tracking, and team collaboration.

System architecture diagrams and flowcharts are created using Draw.io and Lucidchart, ensuring visual clarity in documentation and presentations. Monitoring of cloud-based services is handled using Cloud Logging and Google Cloud Monitoring to ensure observability across all deployed services.

5.4.5 Hardware and Operating Systems

The system is supported by both cloud and local environments. Locally, development is conducted on machines running Windows 11 equipped with NVIDIA RTX 4090 GPUs, offering ample power for local fine-tuning, testing, and UI development.

References

- Chamabouabd. (2023, October 11). *Mistral 7B model architecture explanation*. Medium.
<https://medium.com/@chamabouabd/mistral-7b-model-architecture-explanation-be14e39063f5>
- Google. (2024, February 21). *Gemma explained: Overview of the Gemma model family architectures*. Google Developers Blog. link-[gemma-explained-overview-gemma-model-family-architectures](https://developers.google.com/gemma/overview)
- Grattafiori, A., Dubey, A., Jauhri, A., Pandey, A., Kadian, A., Al-Dahle, A., ... Zhang, J. (2024). *The Llama 3 herd of models* (arXiv:2407.21783). arXiv. <https://doi.org/10.48550/arXiv.2407.21783>
- Hu, E. J., Shen, Y., Wallis, P., Allen-Zhu, Z., Li, Y., Wang, S., ... Catanzaro, B. (2021). *LoRA: Low-Rank Adaptation of Large Language Models* (arXiv:2106.09685). arXiv.
<https://doi.org/10.48550/arXiv.2106.09685>
- Jiang, A., Grave, É., Subramanian, S., Xu, J., Li, X., Albert, P., ... Kocijan, V. (2023). *Mistral 7B* (arXiv:2310.06825). arXiv. <https://doi.org/10.48550/arXiv.2310.068>
- Qwen Team, Yang, A., Yang, B., Hui, B., Zheng, B., Yu, B., Zhou, C., Li, C., Liu, D., Huang, F., Wei, H., Lin, H., Tang, J., Wang, J., Zhang, J., ... Fan, Z. (2024). *Qwen2 technical report* (arXiv:2407.10671). arXiv. <https://doi.org/10.48550/arXiv.2407.10671>

Roque, L. (2025, January 19). *The evolution of LLaMA: From LLaMA 1 to LLaMA 3.1*. Towards Data Science.[The Evolution of Llama: From Llama 1 to Llama 3.1 | Towards Data Science](#)

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., & Polosukhin, I. (2017). *Attention is all you need* (arXiv:1706.03762). arXiv.
<https://doi.org/10.48550/arXiv.1706.03762>